Septum: design docs (v0.1 spec)

2026-11-06

Contents

Septum: handoff for the new project session **2**
- What Septum is . . . 2
- Decisions already made (do not reopen without Khubaib) . . . 2
- What exists . . . 2
- What is NOT real yet . . . 3
- Open items . . . 3
- Suggested first steps . . . 3

1. Problem and research **3**
- The leak nobody types . . . 3
  - Worked example (used in all videos) . . . 3
- Evidence . . . 3
- What Septum is not . . . 3

2. Architecture **4**
- Components . . . 4
- The choice test . . . 4
- Decision table (defaults) . . . 5
- Data flow per action . . . 5

3. CLI spec (v0.1, proposed) **5**
- `septum check --json` . . . 5
- Config: `.septum/septum.toml` . . . 6
- Python API . . . 6

4. Terminal UI spec **6**
- Boot splash (`septum`) . . . 6
- Dashboard: Live tab . . . 6
- ASK modal . . . 7
- Ledger tab . . . 7
- Chambers tab . . . 7
- Policy tab . . . 7
- Exit . . . 7
- Colours . . . 7

5. Integrations **7**
- A. MCP proxy (recommended default) . . . 7
- B. Claude Code `PreToolUse` hook (second wall) . . . 7
- C. Sandbox CLI and library . . . 8
- Which to use . . . 8
- Verified facts (2026-11-06) . . . 8

6. Policy and judge **8**
- Judge backends (pluggable, all local) . . . 8

Policy file `.septum/policy.toml` (proposed) . . . 8
Task scope . . . 9

7. Principles and threat model **9**
Principles (set by Khubaib, 2026-11-06) . . . 9
In scope . . . 9
Out of scope (v0.1) . . . 9
Known hard parts . . . 9

8. Evaluation and roadmap **9**
How to prove it works . . . 9
12-week plan . . . 9
Deliverables . . . 10

9. Brand guide **10**
Name . . . 10
Mark . . . 10
Colour . . . 10
Type . . . 10
Motion . . . 10
Terminal . . . 10
Favicon . . . 11
Banner . . . 11

Septum: handoff for the new project session

Use this file as the first thing a new session reads.

What Septum is

A standalone, agent-agnostic **agent leak firewall** CLI and library. Before every browser action, it checks whether the action would carry a private fact from one site's chamber to another without the user's task asking for it. It then decides ALLOW / NEUTRALIZE / BLOCK / ASK and records the decision in an append-only ledger.

Decisions already made (do not reopen without Khubaib)

- Name: **Septum**.
- Logo: the v3 stone nautilus with one crimson chamber. Approved.
- Animations: intro, regular and loading. Approved.
- Standalone CLI usable by any agent, including from a sandbox. **Not tied to EigenCore**; EigenCore is just one optional judge backend.
- Principle: in sensitive cases, **security and data integrity come before comfort**. Fail closed. No answer = deny.
- Framing confirmed by Khubaib: an evaluator acting for the user, so sensitive data is never picked or used by the browsing agent unless the user's task asks for it.
- Septum has a full terminal UI (splash, dashboard, modal, tabs). It is not only commands.

What exists

- Docs: `README.md` and `docs/01-09`
- Brand: logo SVG/PNG, flat mark, favicons, banner, terminal pixel logo, wordmark, animations, history
- Videos (silent, on-screen text): Part 1 how it works (1:43), Part 2 integrations (1:00), TUI (0:45), plus storyboards
- Generators: `source/` (Python, cairosvg, PIL, pyfiglet, ffmpeg)

What is NOT real yet

No code exists. Every command, the Python `guard()` API, all TUI screens, all output and the ledger are a **design spec**. All demo data is fictional.

Open items

1. The pixel logo in the TUI looked blurry. Redraw it as hand-made pixel art during the build (Khubaib: "we can take care of it when building").
2. Policy tab: not designed.
3. Not made: animated README banner, Lottie/CSS motion, voiceover for the videos.
4. Re-verify the AgentTell figures against the paper (they came from search excerpts).
5. Unverified: whether Codex hooks fire on MCP calls; the Windows `PreToolUse` MCP bug.
6. License choice; whether to reserve `septum` on PyPI now.
7. Repo: none yet. Khubaib commits as **Khubaib Nazeer / Khubaib7-del**.

Suggested first steps

1. Create the repo, then add the README, docs and brand files.
2. Week 1-2 milestone: reproduce the AgentTell baseline.
3. Scaffold `septum check --json` and the rules judge first; it is the core every door uses.

1. Problem and research

The leak nobody types

AI agents now act for us in browsers. In one session an agent may read your health portal, your bank and your email, then fill a form on an unrelated site. Most privacy tools watch for **text** going out, such as a card number pasted into a field. Agents leak in a quieter way: through **actions**.

Worked example (used in all videos)

1. Task 1: *"Refill my prescription."* The agent reads `myhealth.portal` and sees **Diagnosis: Type 2 diabetes**.
2. Task 2, same session: *"Get me a car insurance quote."* On `insure.co` there is an **optional** field, "Pre-existing medical conditions", with options `None`, `Prefer not to say` and `Diabetes`.
3. Trying to be helpful, the agent picks **Diabetes**. No secret was typed, yet the choice gave it away. The user never asked for it, and the quote may now cost more.

Septum's job is to stop step 3, rewrite it to `Prefer not to say`, tell the agent why, and log it.

> The demo names (MyHealth Portal, insure.co, Islamabad, Toyota Corolla 2019, phone numbers, timestamps) are fictional.

Evidence

- **AgentTell** (arXiv 2609.32915, https://arxiv.org/html/2609.32915v1) is a benchmark for action-based privacy leaks by web agents. Figures used in our material:
  - action-based leaks in **61.1%** of sessions
  - **56.7%** of sessions still leaked when the agent was told not to share
  - **34.5%** of leaking sessions included a false assurance that nothing was shared
- AgentTell measures the problem. It ships no defence, and that gap is where Septum sits.

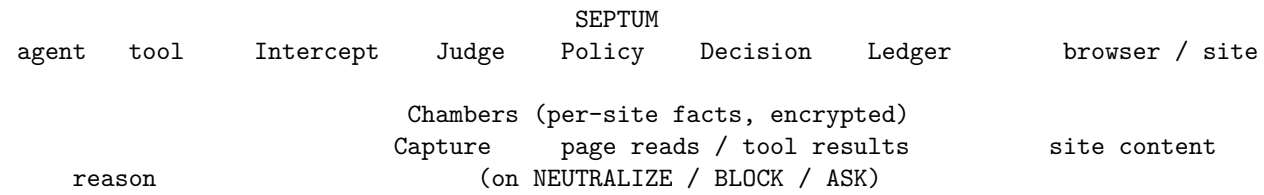
  Provenance caveat: these figures came from search excerpts during idea research, not a full read of the paper. Re-read the paper and confirm each number and its denominator (especially the 34.5%) before you publish anything.

What Septum is not

- It is not a DLP regex scanner. Text matching is only one of its checks.

- It is not a prompt telling the agent to behave. The data above shows prompting is not enough.
- It is not tied to any agent, model or runtime.

2. Architecture



Components

Component	Job	v0.1 plan
Capture	Pull sensitive facts out of what the agent reads (page text, tool results, form values) and file each one in that site's chamber with its origin and category.	Rules plus NER plus a small local model. Categories: HEALTH, MONEY, IDENTITY, CONTACT, LOCATION, RELATIONSHIP, CREDENTIAL.
Chambers	One store per origin (eTLD+1). Facts are stored redacted in the UI and encrypted at rest. A special `task` chamber holds facts the user's own task supplies.	SQLite plus encryption at rest, local only.
Intercept	Pause every outgoing action before it runs: `type`, `select`, `click` (on choice controls), `submit`, `goto` with query params, uploads.	MCP proxy, hook, CLI and library entry points (see 05).
Judge	Ask: *does this action reveal a fact from another chamber?* Three tests: text match (exact, fuzzy, semantic), the **choice test** (does choosing this option over the alternatives reveal a fact?), and the URL/param test.	Rules first, then a local model for the hard cases. Target < 50 ms on a CPU for the common path.
Policy	Weigh what the judge found against sensitivity, the user's **task scope** and whether the field is required.	TOML policy, fail-closed defaults.
Decision	ALLOW - NEUTRALIZE (swap to a neutral option or blank optional field) - BLOCK - ASK (user approves, with auto-deny on timeout).	
Ledger	Append-only log of every cross-chamber flow: time, from, to, fact, sensitivity, decision, who decided, reason.	JSONL plus SQLite index. The agent cannot claim "nothing was shared" unless the ledger agrees.

The choice test

For a `select`, radio or checkbox action with options `O = {o1..on}` and chosen `oc`:

1. Find facts `F` in **other** chambers related to the field's label.
2. For each fact, ask whether `oc` is more consistent with it than a neutral option (`None`, `Prefer not to say`, blank) would be.

3. If yes and the task does not need it, the choice *reveals* the fact. The verdict is NEUTRALIZE to the most neutral valid option, or BLOCK if the field is required and there is no neutral option.

Decision table (defaults)

Fact found from another chamber?	Sensitivity	Task needs it?	Field	Decision
No	n/a	n/a	n/a	ALLOW
Yes	any	yes, stated explicitly	any	ALLOW (logged)
Yes	LOW/MED	plausibly	required	ASK
Yes	HIGH	not stated	optional	NEUTRALIZE
Yes	HIGH	not stated	required	BLOCK, or ASK if the policy allows
Judge error or timeout	n/a	n/a	n/a	BLOCK (fail-closed)

Data flow per action

1. The agent calls a tool, e.g. `browser_select_option(ref, "Diabetes")`.
2. Intercept resolves the target: site, field label, options, required flag.
3. The judge runs its tests against the chambers and returns findings plus a latency figure.
4. Policy maps the findings to a decision.
5. The decision is applied. ALLOW forwards the call unchanged. NEUTRALIZE forwards a rewritten call and returns a note to the agent. BLOCK returns an error with the reason. ASK pauses for the user.
6. A ledger entry is written.

3. CLI spec (v0.1, proposed)

Single binary/entry point: `septum`. Running it with no subcommand opens the TUI (see 04).

Command	Purpose
`septum`	Open the TUI dashboard for the active session
`septum init`	Create `.septum/` with `septum.toml`, `policy.toml` and a local judge auto-detect
`septum mcp --upstream "<cmd>"`	Run as an MCP server that wraps an upstream MCP server (e.g. Playwright MCP) and checks every tool call
`septum check [--json] [--hook claude]`	Judge one action read from stdin. Exit 0 allow, exit 2 deny (hook mode), JSON decision on stdout
`septum run -- <cmd...>`	Launch an agent process with its browser wrapped over CDP, with no code changes
`septum ledger [--last N] [--json]`	Show or export ledger entries
`septum chambers [--reveal <fact>]`	List chambers. Reveal asks for OS authentication first
`septum policy [edit|test]`	Show, edit or dry-run the policy
`septum judge [bench]`	Show the judge backend; benchmark latency

`septum check --json`

Input:

```
{"site":"insure.co","action":"select","field":"Pre-existing conditions",
 "required":false,"value":"Diabetes",
 "options":["None","Prefer not to say","Diabetes"],
 "task":"car insurance quote"}
```

Output:

```
{"decision":"neutralize","replace":"Prefer not to say",
 "fact":"myhealth.portal/diagnosis","category":"HEALTH","severity":"high",
 "reason":"choice reveals a health fact the task does not need","judge_ms":38}
```

Decisions: `allow | neutralize | block | ask`. Exit codes: `0` allow or neutralize (with a rewrite), `2` block or deny, `3` ask pending (non-interactive callers should treat this as deny), `1` internal error (callers must treat it as deny).

Config: `.septum/septum.toml`

```
[judge]
backend = "auto"      # "ollama" | "llamacpp" | "openai-compatible" | "eigencore" | "rules"
model   = "auto"
cloud   = false       # facts never leave the machine

[ask]
timeout_s = 60        # no answer = deny
```

Python API

```
from septum import guard
page = guard(await browser.new_page(), task="car insurance quote")
```

4. Terminal UI spec

See `video/septum-tui.mp4` and `video/storyboards/tui-storyboard.png`. Target: Python **Textual** (built on Rich), on truecolor terminals. Reference grid: 148x40 cells.

Boot splash (`septum`)

- Pixel nautilus drawn in truecolor half-blocks (`half-block` with fg and bg colours, so 2 px per cell). Source: `brand/terminal/septum-pixel-44x37.png`. It reveals outward from the eye in about 1.4 s.
- Wordmark in figlet **ANSI Shadow** (`brand/terminal/wordmark-ansi-shadow.txt`): `block` in off-white, box-drawing shadow in dim grey.
- Tagline: `agent leak firewall - every site is a chamber - v0.1.0`
- Boot checks, each with a spinner and then ok: policy, judge, upstream, chambers, ledger.
- Fallbacks: on a 256-colour terminal, quantise the pixel art. With no Unicode, show a plain `SEPTUM` text banner. `--no-splash` and the `SEPTUM_NO_SPLASH` env var skip the splash. Below 100x30 cells, skip the logo.
- Known issue: the pixel logo looked slightly blurry in the video. Redraw it by hand as true pixel art (not a downscale) during the build.

Dashboard: Live tab

- **Header bar:** `* septum 0.1  task "..."  upstream ...  judge local - 38 ms  mode FAIL-CLOSED * guarding  clock`
- **Tabs:** `1 Live  2 Ledger  3 Chambers  4 Policy`
- **Chambers panel:** one row per site with a sensitivity badge (HIGH red, MED amber, TASK green) and facts shown as `name ****`. The row is highlighted while one of its facts is under judgement.
- **Session panel:** counts (checked, allowed, neutralized, asked you, blocked) and an actions/sec sparkline.
- **Live actions:** time, verdict badge, verb, value and target. The selected row has a red edge. A neutralized row shows `rewritten to "..."`.
- **Inspector:** action, target, checks revealed one at a time (text match, choice test with source fact, sensitivity, task scope), then the verdict block and judge latency.
- **Ledger strip:** the last flows.
- **Toast line** and **key hints:** `up/down select - enter inspect - a allow - d deny - 1-4 tabs - / filter - e export - q quit`

ASK modal

The background dims. The modal has a red border titled *SEPTUM NEEDS YOUR DECISION* and shows what the agent wants to do, where, the source chamber, the sensitivity, the task and why Septum is asking. Keys: `a` allow once - `t` allow for this task - `d` deny. A countdown bar shows auto-deny (default 60 s). *No answer = deny.*

Ledger tab

A full table: time, from, to, fact, sensitivity, decision, by, reason. It shows `unauthorised cross-site flows this session: N` and the export hint.

Chambers tab

A 2x2 grid of chamber cards with redacted facts, categories and a crossing summary. `r` reveals a fact after OS authentication.

Policy tab

Not designed yet.

Exit

`q` prints a session summary to the shell: counts, unauthorised flows and the ledger path.

Colours

Background `#0B0D12` - text `#F3F1EC` - muted `#9AA0AC` - dim `#5E6573` - lines `#2A2F3B` - allow `#4CC38A` - neutralize `#E2A33B` - ask `#6CB6FF` - block/accent `#E5484D`. Font in videos: JetBrains Mono.

5. Integrations

Septum is a standalone CLI and library. Every door goes through the same chambers, judge, policy and ledger.

A. MCP proxy (recommended default)

`agent -> septum mcp -> Playwright MCP -> Chromium`. The agent sees the same tools, but every call crosses Septum first. Septum can **rewrite** a choice, not only block it.

```
claude mcp add septum -- septum mcp --upstream "npx @playwright/mcp@latest"
codex  mcp add septum -- septum mcp --upstream "npx @playwright/mcp@latest"
```

```
# ~/.codex/config.toml  (or .codex/config.toml in a trusted project)
[mcp_servers.septum]
command = "septum"
args = ["mcp", "--upstream", "npx @playwright/mcp@latest"]
```

B. Claude Code `PreToolUse` hook (second wall)

```
{ "hooks": { "PreToolUse": [{
    "matcher": "mcp__playwright__.*",
    "hooks": [{ "type": "command", "command": "septum check --hook claude" }]
}] } }
```

- Exit 0 allows the call. Exit 2 blocks it, and stderr goes back to Claude as the reason. Exit 1 does **not** block, so Septum's hook mode must never exit 1 on error; it should exit 2.
- Deny/ask permission rules still apply on top of the hook.
- Windows caveat (unconfirmed): a GitHub issue (anthropics/claude-code #85479) reports `PreToolUse` not firing for MCP tools on Windows, and another report (#74203) conflicts with it. Rely on the proxy there.

C. Sandbox CLI and library

```
echo '{...action json...}' | septum check --json
septum run -- python my_agent.py      # wraps the browser over CDP, no code changes
```

```
from septum import guard
page = guard(await browser.new_page(), task="car insurance quote")
```

Which to use

Agent	Use	Note
Claude Code	MCP proxy	add the hook as belt and braces
Codex	MCP proxy	`codex mcp add` or config.toml. Whether Codex's hook events cover MCP calls is **unverified**
Playwright / browser-use agent	Python guard	or `septum run`
Any sandboxed agent with a shell	`septum check --json`	

Verified facts (2026-11-06)

- Claude Code hooks: `PreToolUse` runs before tool calls and matchers can target `mcp__<server>__.*`. Exit 2 blocks the call and feeds stderr back. Source: Claude Code hooks docs.
- Codex MCP: `codex mcp add`, `[mcp_servers.<name>]` in `~/.codex/config.toml`, project config only in trusted projects, remote servers via `--url`. Source: OpenAI Codex MCP docs.

6. Policy and judge

Judge backends (pluggable, all local)

`rules` (no model: fastest and strictest) - `ollama` - `llamacpp` - `openai-compatible` (any local server) - `eigencore` (optional; Khubaib's own CPU-first runtime). `cloud = false` by default, so facts never leave the machine.

The judge answers one narrow question per action, so a small quantised model is enough. Rules handle the common path; the model handles semantic and choice cases.

Policy file `.septum/policy.toml` (proposed)

```
mode = "fail-closed"

[sensitivity]
HEALTH = "high"
MONEY = "high"
CREDENTIAL = "high"
IDENTITY = "high"
CONTACT = "medium"
LOCATION = "medium"
RELATIONSHIP = "high"

[defaults]
high_optional  = "neutralize"
high_required  = "block"        # or "ask"
medium_needed  = "ask"
judge_error    = "block"

[ask]
timeout_s = 60
```

```toml
on_timeout = "deny"

[[allow]]                          # explicit user grants
fact = "profile/email"
to = "insure.co"
scope = "task"
```

Task scope

The task string (e.g. "get a car insurance quote") is the contract. Facts the task names go into the `task` chamber and flow freely. Anything else must be justified by the policy or approved by the user.

7. Principles and threat model

Principles (set by Khubaib, 2026-11-06)

1. **Security over comfort.** In sensitive cases, confidentiality and data integrity come before convenience. Extra prompts and broken optional steps are acceptable costs.
2. **Fail closed.** Judge error, timeout or no answer from the user means deny.
3. **Septum never guesses on the user's behalf.** If the task did not ask for a sensitive fact, the agent may not volunteer it, however helpful it looks.
4. **Honest record.** The ledger is the source of truth for what was shared. The agent's own claims are not.
5. **Local only.** Facts and judgements stay on the machine.
6. **Agent-agnostic.** No dependency on one agent, model or runtime.

In scope

- Action-based leaks: typed text, option choices, checkboxes, submits, URL parameters, uploads.
- Cross-site flows within one agent session.
- False assurances, which the ledger contradicts.

Out of scope (v0.1)

- Malicious agents deliberately evading Septum, e.g. encoding facts steganographically.
- Leaks through channels Septum does not wrap, such as raw HTTP from the agent or other tools.
- Prompt injection defence in general. This is related but separate.

Known hard parts

- Over-blocking breaks tasks, so measure task success alongside leak rate.
- Inferring task scope is subtle.
- Judge speed and accuracy on a laptop CPU.
- Chamber identity for SSO and shared domains.

8. Evaluation and roadmap

How to prove it works

Run the AgentTell suite **with and without** Septum and measure: - leak rate (primary) - task success rate, i.e. whether blocking broke useful work - false-assurance rate, checked against the ledger - added latency per action (p50/p95) - ASK prompts per session, as a usability cost

12-week plan

Weeks	Milestone
1-2	Reproduce the AgentTell baseline with a Playwright/browser-use agent
3-5	Chambers and fact capture
6-8	Judge: text, choice and URL tests; rules plus a local model
9-10	Policy engine, decisions, ledger. MCP proxy and `septum check`
11	TUI (Textual), Claude Code hook, `septum run`
12	Full evaluation and write-up (workshop paper target)

Deliverables

An open-source CLI and library, measured results on a public benchmark, and a paper draft.

9. Brand guide

Name

Septum: the wall between two chambers, as in a nautilus shell or the heart. Each site is a chamber, and Septum is the wall. Availability checked 2026-11-06: `septum` is free on PyPI, **taken on npm**, and no GitHub agent repos use it. Backups: `keepmum` (free on PyPI, npm and GitHub). `hushgate` is free on PyPI but taken on npm. Bulkhead, Airlock and Tacit are taken.

Mark

A solid stone 2.5D nautilus shell with thin engraved curved septa and **one crimson chamber**, the fact kept in its room. Approved 2026-11-06. - `brand/logo/septum-mark.svg` (transparent), `-dark-bg`, `-light-bg`, and PNGs at 128-1024 - `septum-mark-flat.svg` for small sizes and single-colour use - Rejected directions are kept in `brand/history/`: rounded rectangles (looked like UI buttons) and chunky separated blocks (looked like a spine).

Colour

Role	Hex
Ink (background)	`#0B0D12`
Off-white	`#F3F1EC`
Accent crimson (one accent only)	`#E5484D`
Backup accent plum	`#8E6CEF`

TUI status colours are listed in 04.

Type

Inter (UI and wordmark in graphics) and JetBrains Mono (CLI and code). Both are under the SIL Open Font License.

Motion

`brand/animation/`: **intro** (4.2 s, with wordmark), **regular** (2 s) and **loading** (seamless 1.6 s loop), each as MP4 and GIF.

Terminal

Half-block pixel logo and ANSI Shadow wordmark in `brand/terminal/`. Redraw the pixel logo by hand at build time; the current one is a downscale and reads slightly soft.

Favicon

`brand/favicon/`. Sizes 16-48 use the flat mark, which reads as a silhouette plus the red chamber at that size. Includes `favicon.ico` and `apple-touch-icon.png`.

Banner

`brand/banner/septum-readme-banner.png` (1280x320). Not made yet: an animated README banner.